

 [Go back to the homepage \(https://flaviocopes.com/\)](https://flaviocopes.com/)

Custom events in JavaScript

Published Jun 28 2020

[Join the 2022 Full-Stack Web Dev Bootcamp! \(https://bootcamp.dev\)](https://bootcamp.dev)

Many of the code we write involves reacting to events. I/O events like mouse clicks or keyboard events. Network events, when you listen to an HTTP call.

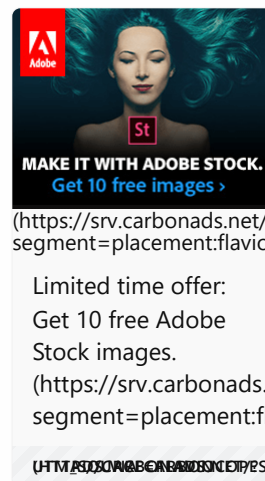
Those are what I call built-in events.

In JavaScript we can create custom events, and the way it works changes in the browser and in Node.js.

In the frontend we use the Event object which is provided by the browser:

```
const anEvent = new Event('start');
```

You can trigger the event using



```
document.dispatchEvent(anEvent)
```

and when this happens, the event listener is triggered:

```
document.addEventListener('start', event => {  
  console.log('started!')  
})
```

You can send custom data using the `CustomEvent` built-in object instead of `Event`, which accepts an object as the second parameter:

```
const anotherEvent = new CustomEvent('start', {  
  detail: {  
    color: 'white'  
  }  
})
```

Using `CustomEvent`, in the event listener you can ask the data to the event object using `event.detail` (you can't use another property):

```
document.addEventListener('start', event => {  
  console.log('started!')  
  console.log(event.detail)  
})
```

On the backend side, Node offers us the option to build a similar system using the `events` module.

This module, in particular, offers the `EventEmitter` class, which we'll use to handle our events.

You initialize that using

```
const EventEmitter = require('events')
const eventEmitter = new EventEmitter()
```

This object exposes, among many others, the `on` and `emit` methods.

- `emit` is used to trigger an event
- `on` is used to add a callback function that's going to be executed when the event is triggered

For example, let's create a `start` event, and as a matter of providing a sample, we react to that by just logging to the console:

```
eventEmitter.on('start', () => {
  console.log('started')
})
```

When we run

```
eventEmitter.emit('start')
```

the event handler function is triggered, and we get the console log.

You can pass arguments to the event handler by passing them as additional arguments to `emit()`:

```
eventEmitter.on('start', (number) => {  
  console.log(`started ${number}`)  
})  
  
eventEmitter.emit('start', 23)
```

Multiple arguments:

```
eventEmitter.on('start', (start, end) => {  
  console.log(`started from ${start} to ${end}`)  
})  
  
eventEmitter.emit('start', 1, 100)
```

Watch my latest YouTube video (<https://www.youtube.com/watch?v=VquX088paco>) !

FREE EBOOKS 

JavaScript
(<https://flaviocopes.com/page/javascript-handbook>)

Python (<https://flaviocopes.com/page/python-handbook/>)